

Advanced SQL Master Handbook

This handbook is designed as a complete SQL revision guide for interviews, placements, internships, and practical database development. Topics Included: - Basic Queries - Aggregate Functions - GROUP BY & HAVING - INNER JOIN - LEFT JOIN - SELF JOIN - Placement-Level SQL Problems - Real Interview Questions - Detailed Concept Explanations Each section progresses from beginner level to advanced interview-level difficulty.

1. BASIC SQL QUERIES

Basic SQL queries are used to retrieve data from database tables. These concepts form the foundation of SQL. Every advanced SQL query depends on understanding SELECT, WHERE, ORDER BY, DISTINCT, LIMIT, LIKE, BETWEEN, and IN.

Level 1: Display all employee names.

```
SELECT name FROM employees;
```

Explanation: SELECT retrieves specific columns from a table. This query displays only employee names.

Level 2: Display employee names and salaries.

```
SELECT name, salary FROM employees;
```

Explanation: Multiple columns can be selected together using commas.

Level 3: Find employees earning more than 50000.

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

Explanation: WHERE filters rows before results are displayed.

Level 4: Find employees from HR department.

```
SELECT * FROM employees WHERE department='HR';
```

Explanation: The WHERE clause can filter rows using equality conditions.

Level 5: Find employees whose salary is between 40000 and 80000.

```
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 80000;
```

Explanation: BETWEEN checks values within a range inclusively.

Level 6: Find employees working in IT, HR, or Sales.

```
SELECT * FROM employees WHERE department IN ('IT', 'HR', 'Sales');
```

Explanation: IN is used to match multiple possible values efficiently.

Level 7: Find students whose names start with A.

```
SELECT * FROM students WHERE name LIKE 'A%';
```

Explanation: LIKE performs pattern matching. % represents multiple characters.

Level 8: Display top 5 highest paid employees.

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

Explanation: ORDER BY sorts rows while LIMIT restricts result count.

Interview-Level Question

Find the second highest salary.

```
SELECT MAX(salary)
FROM employees
WHERE salary < (
    SELECT MAX(salary) FROM employees
);
```

Detailed Explanation: The inner query finds highest salary. The outer query excludes it and finds the maximum remaining salary.

2. AGGREGATE FUNCTIONS

Aggregate functions summarize data and return a single value. They are used heavily in reporting systems, dashboards, analytics, and interview problems. Main functions: COUNT(), SUM(), AVG(), MIN(), MAX()

Level 1: Count total employees.

```
SELECT COUNT(*) FROM employees;
```

Explanation: COUNT(*) counts all rows.

Level 2: Find total salary paid.

```
SELECT SUM(salary) FROM employees;
```

Explanation: SUM adds all numeric values together.

Level 3: Find average employee salary.

```
SELECT AVG(salary) FROM employees;
```

Explanation: AVG calculates mean value.

Level 4: Find highest salary.

```
SELECT MAX(salary) FROM employees;
```

Explanation: MAX returns largest value.

Level 5: Find lowest marks among students.

```
SELECT MIN(marks) FROM students;
```

Explanation: MIN returns smallest value.

Level 6: Find average salary of IT employees.

```
SELECT AVG(salary) FROM employees WHERE department='IT';
```

Explanation: Aggregate functions can work with WHERE conditions.

Level 7: Find total orders from Delhi customers.

```
SELECT SUM(amount) FROM orders WHERE city='Delhi';
```

Explanation: WHERE filters rows before aggregation happens.

Interview-Level Question

Find departments with average salary greater than 70000.

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 70000;
```

Detailed Explanation: GROUP BY creates department groups while HAVING filters aggregated results.

3. GROUP BY & HAVING

GROUP BY divides rows into groups. Aggregate functions then work on each group separately. HAVING filters grouped data after aggregation. Difference: WHERE -> filters rows before grouping HAVING -> filters groups after grouping

Level 1: Count employees department-wise.

```
SELECT department, COUNT(*) FROM employees GROUP BY department;
```

Explanation: Rows are grouped by department before COUNT executes.

Level 2: Find average salary department-wise.

```
SELECT department, AVG(salary) FROM employees GROUP BY department;
```

Explanation: Each department gets its own average salary.

Level 3: Find highest salary department-wise.

```
SELECT department, MAX(salary) FROM employees GROUP BY department;
```

Explanation: MAX works independently on each department group.

Level 4: Find cities having more than 5 students.

```
SELECT city, COUNT(*)
FROM students
GROUP BY city
HAVING COUNT(*) > 5;
```

Explanation: HAVING filters grouped rows based on aggregate results.

Level 5: Find departments with minimum salary greater than 50000.

```
SELECT department, MIN(salary)
FROM employees
GROUP BY department
HAVING MIN(salary) > 50000;
```

Explanation: Only departments satisfying aggregate condition are returned.

Interview-Level Question

Find department-wise employee count, average salary, highest salary, and lowest salary.

```
SELECT department,
COUNT(*) AS total_employees,
AVG(salary) AS avg_salary,
MAX(salary) AS highest_salary,
MIN(salary) AS lowest_salary
FROM employees
GROUP BY department;
```

Detailed Explanation: This is a common business reporting query combining multiple aggregate functions.

4. JOINS

JOIN combines data from multiple tables. Joins are the most important topic for SQL interviews. Types: - INNER JOIN - LEFT JOIN - RIGHT JOIN - FULL JOIN - SELF JOIN

Level 1: Display employee names with department names.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

Explanation: INNER JOIN returns only matching rows from both tables.

Level 2: Display all customers with orders.

```
SELECT c.customer_name, o.amount
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Explanation: LEFT JOIN keeps all rows from left table.

Level 3: Find customers who never placed orders.

```
SELECT c.customer_name
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Explanation: NULL values indicate unmatched rows after LEFT JOIN.

Level 4: Display employees with manager names.

```
SELECT e.name AS employee_name,  
m.name AS manager_name  
FROM employees e  
LEFT JOIN employees m  
ON e.manager_id = m.emp_id;
```

Explanation: SELF JOIN joins a table with itself.

Level 5: Find total employees department-wise using JOIN.

```
SELECT d.department_name,  
COUNT(e.emp_id)  
FROM departments d  
LEFT JOIN employees e  
ON d.department_id = e.department_id  
GROUP BY d.department_name;
```

Explanation: JOIN + GROUP BY is heavily used in reporting systems.

Interview-Level Question

Find employees earning greater than department average salary.

```
SELECT e.name, e.salary  
FROM employees e  
WHERE e.salary > (  
    SELECT AVG(salary)  
    FROM employees  
    WHERE department_id = e.department_id  
);
```

Detailed Explanation: This uses correlated subqueries and is frequently asked in interviews.

Most Important SQL Interview Questions

Question Type
Second Highest Salary
Nth Highest Salary
Find Duplicate Records
Delete Duplicate Records
Employees earning more than department average
Customers who never ordered
Manager-Employee hierarchy
Top 3 salaries department-wise
Departments with highest average salary
Find running total using window functions

SQL Execution Order

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Most Important Topics for Placements

1. JOINS
2. GROUP BY + HAVING
3. Aggregate Functions
4. Subqueries
5. Window Functions

Recommended Next Topics

- Subqueries
- Window Functions
- CTE (Common Table Expressions)
- Indexing
- Query Optimization